Postgres ClickHouse Data Stack: A Walkthrough

This is a walkthrough where you can setup Postgres, ClickHouse, pg_clickhouse and PeerDB separately and wire them together, to get a feel for this data stack.

Clone this repository

1. Run:

```
git clone https://github.com/ClickHouse/postgres-clickhouse-stack.git
```

2. Enter the directory

```
cd postgres-clickhouse-stack
```

Start the services

This will spin up Postgres, ClickHouse and PeerDB along with a ClickHouse playground.

```
./run.sh start
```

After it runs you should see the service URLs and commands:

```
==========================================
   Stack is up and running!
==========================================

Sample App:
   URL: http://localhost:18080
   Seed data: ./run.sh seed

PeerDB UI:
```

```
   URL: http://localhost:13000

ClickHouse UI (HTTP Interface):
   URL: http://localhost:18123/play

ClickHouse Client:
   Command: ./run.sh clickhouse

PostgreSQL:
   Command: ./run.sh psql
```

Setting up Postgres

1. Run the following command to PSQL into Postgres:

   ```
   ./run.sh psql
   ```

   You'll see this:

   ```
   amogh@Amoghs-MacBook-Pro postgres-clickhouse-stack %
   ./run.sh psql
   psql (18.2 (Debian 18.2-1.pgdg13+1))
   Type "help" for help.

   postgres=#
   ```

2. First let's examine the `expenses` table that we have:

   ```
   \d public.expenses;
   ```

   You will see the schema of the table:

```
postgres=# \d public.expenses;
                                          Table
"public.expenses"
   Column    |            Type             | Collation |
Nullable |               Default
-------------+-----------------------------+-----------
+----------+--------------------------------------
 id          | integer                     |           |
not null | nextval('expenses_id_seq'::regclass)
 description | text                        |           |
not null |
 amount      | numeric(10,2)               |           |
not null |
 category    | character varying(100)      |           |
         |
 date        | date                        |           |
not null | CURRENT_DATE
 created_at  | timestamp without time zone |           |
         | CURRENT_TIMESTAMP
Indexes:
    "expenses_pkey" PRIMARY KEY, btree (id)
```

3. Run a COUNT:

```
SELECT COUNT(*) FROM public.expenses;
```

You will see it has 0 rows:

```
postgres=# SELECT COUNT(*) FROM public.expenses;
 count
-------
     0
(1 row)
```

Ingesting data

4. Let's seed the above table. Open a new terminal in this `postgres-`

`clickhouse-stack` repository root and run the below command:

```
./run.sh seed 20000000
```

It should take a minute. You should see this at the end:

```
...
Progress: 92.5% - 18,500,000 rows - 407,687 rows/s -
45.4s elapsed - ETA: 4s
Progress: 95.0% - 19,000,000 rows - 408,629 rows/s -
46.5s elapsed - ETA: 2s
Progress: 97.5% - 19,500,000 rows - 409,363 rows/s -
47.6s elapsed - ETA: 1s
Progress: 100.0% - 20,000,000 rows - 407,697 rows/s -
49.1s elapsed - ETA: 0s
Seeding completed!
Total rows: 20,000,000
Time taken: 49.1s (0.8 min)
Average: 407,332 rows/second
```

5. PSQL into Postgres again. Let's check the count, should see 20 million

```
postgres=# SELECT COUNT(*) FROM expenses;
  count
----------
 20000000
(1 row)
```

6. Feel free to run some more queries to play around!

Setting up ClickHouse

1. Run the following command to shell into ClickHouse server. This is analagous to PSQLing into a Postgres server.

```
./run.sh clickhouse
```

You should see this:

```
amogh@Amoghs-MacBook-Pro postgres-clickhouse-stack %
./run.sh clickhouse
ClickHouse client version 25.8.18.1 (official build).
Connecting to localhost:9000 as user default.
Connected to ClickHouse server version 25.8.18.

Warnings:
 * Delay accounting is not enabled, OSIOWaitMicroseconds
will not be gathered. You can enable it using `sudo sh -
c 'echo 1 > /proc/sys/kernel/task_delayacct'` or by
using sysctl.

ch_postgres :)
```

2. Create a new database called `expense`. This is the database which PeerDB and pg_clickhouse will be looking at.

```
CREATE DATABASE IF NOT EXISTS expense;
```

You should see:

```
CREATE DATABASE expense

Query id: aea717b0-c93a-4d12-98b5-49f3e01154d2

Ok.

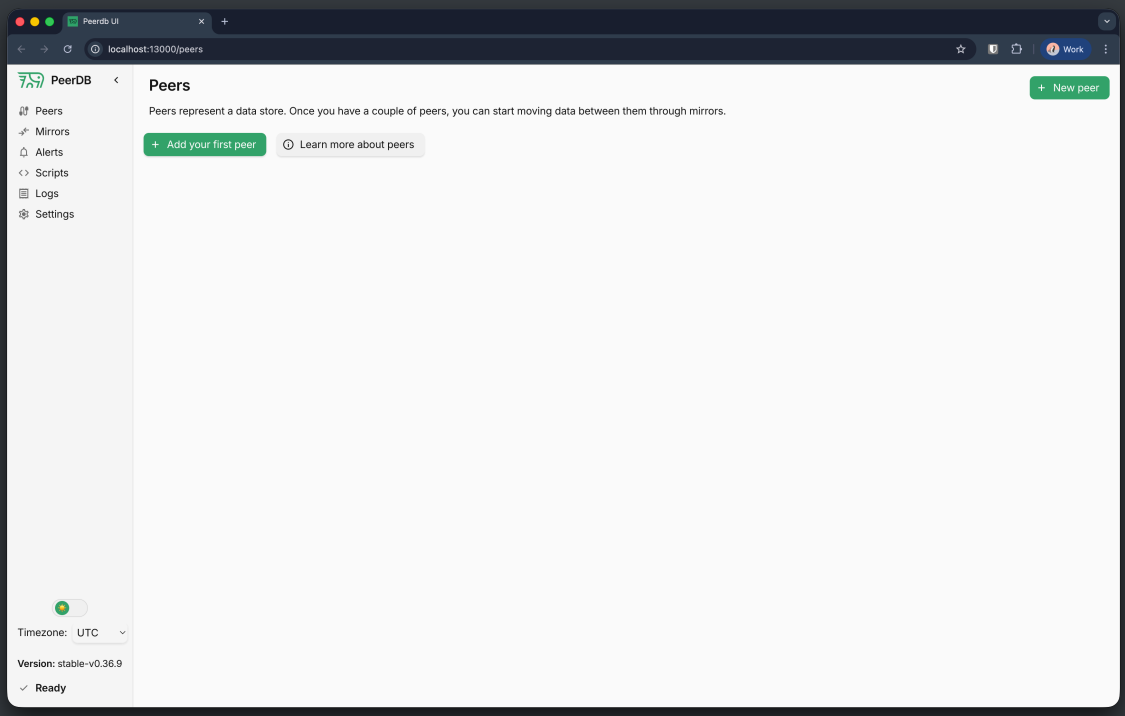
0 rows in set. Elapsed: 0.025 sec.
```

Setting up PeerDB

Here we are going to create **peers** for Postgres and ClickHouse. **Peer** is a PeerDB term, it means a representation of a data store.

1. Head over to http://localhost:18080 in your browser. You should see this

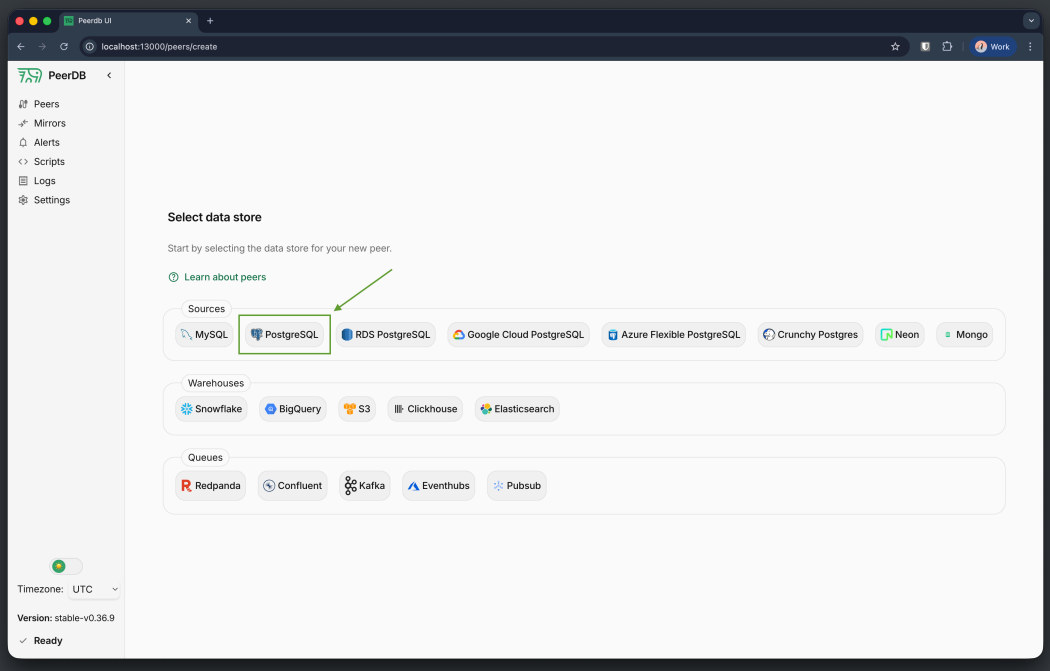
page render:



2. Click on **Add your first peer**

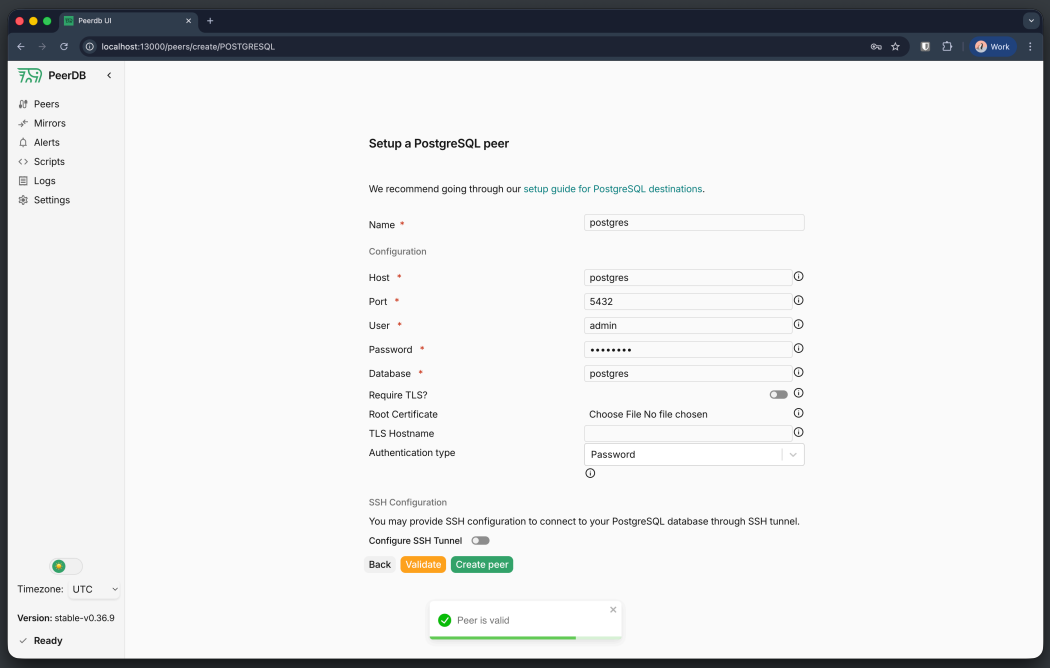
Creating the Postgres peer

1. Select the `PostgreSQL` tile as shown below:

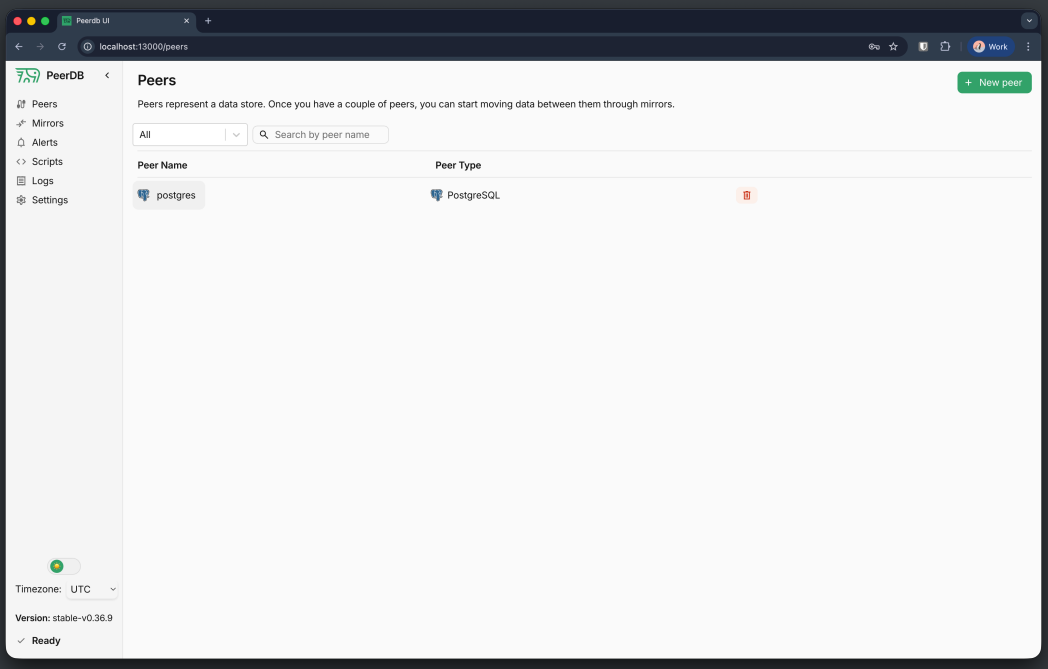


2. Enter the following form details:

```
Name: postgres
Host: postgres
Port: 5432
User: admin
Password: password
Database: postgres
```

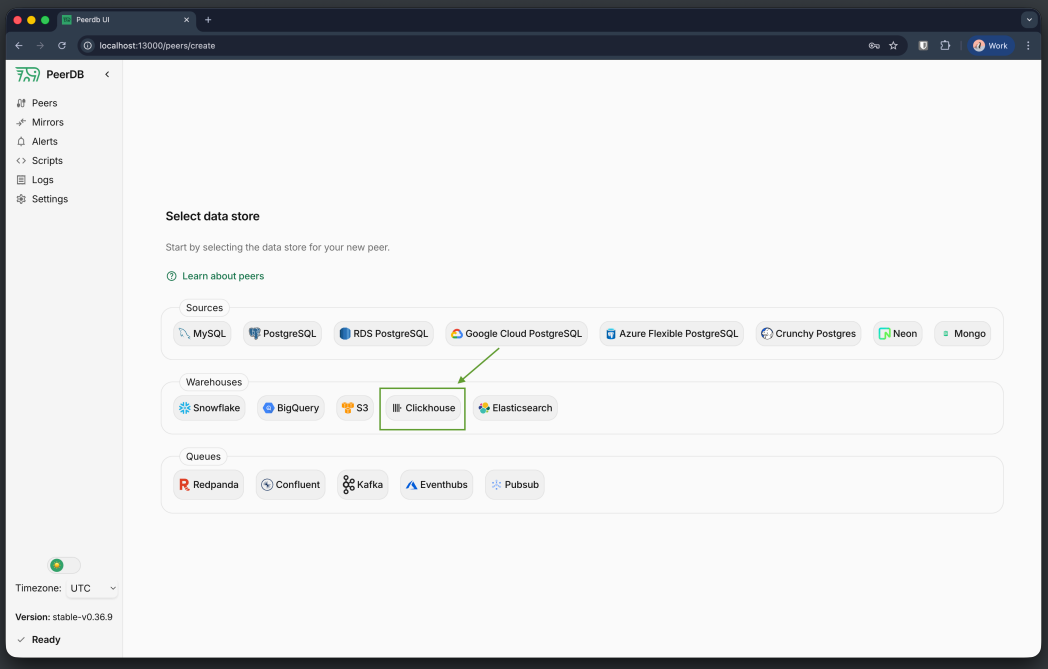


3. Click on **Validate**. It should say the peer is valid.
4. Click on **Create**. You should the peer created:



Create the ClickHouse peer

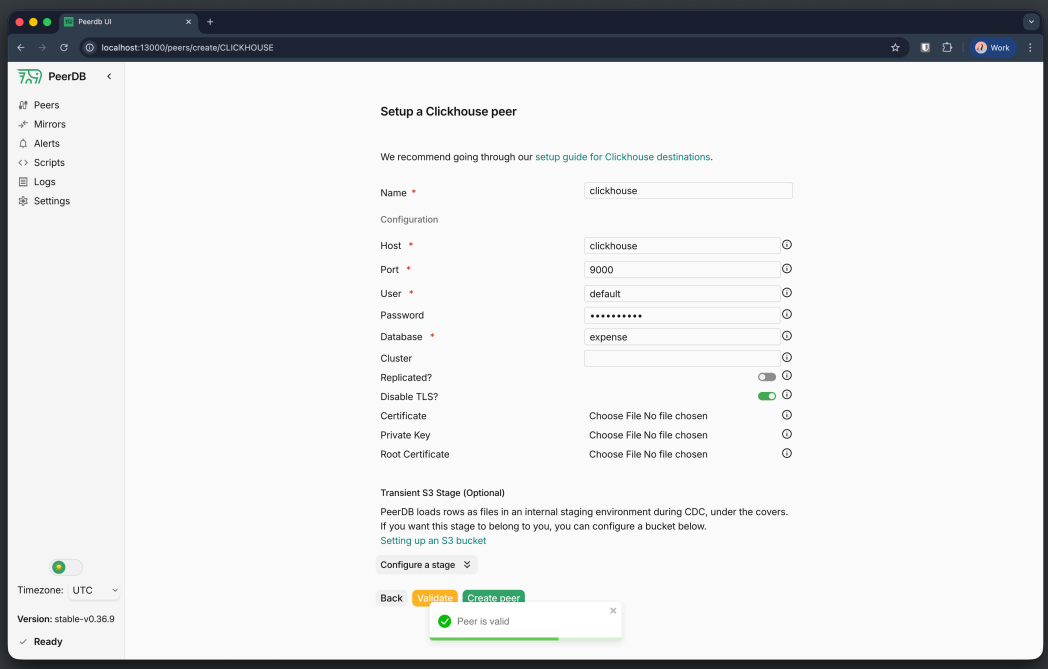
1. Go to http://localhost:13000/peers and click on **New peer**
2. Click on the `ClickHouse` tile found in the `Warehouses` section:



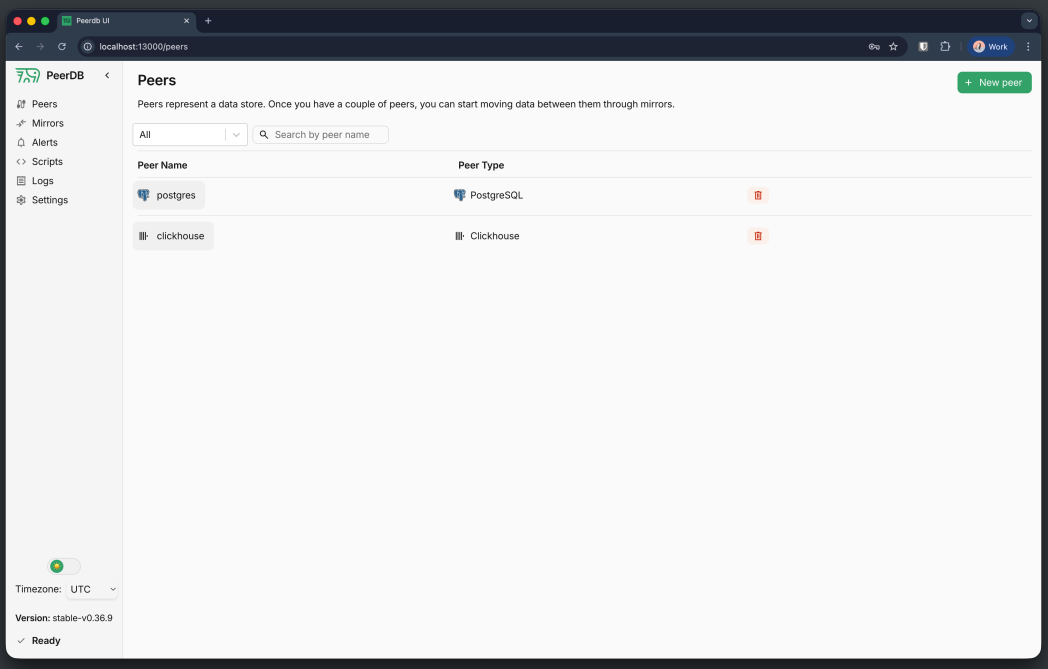
3. Fill in the following form details:

```
Name: clickhouse
Host: clickhouse
Port: 9000
User: default
Password: clickhouse
Database: expense
Disable TLS?: true
```

Make sure to turn on the Disable TLS switch. The form should look like this:



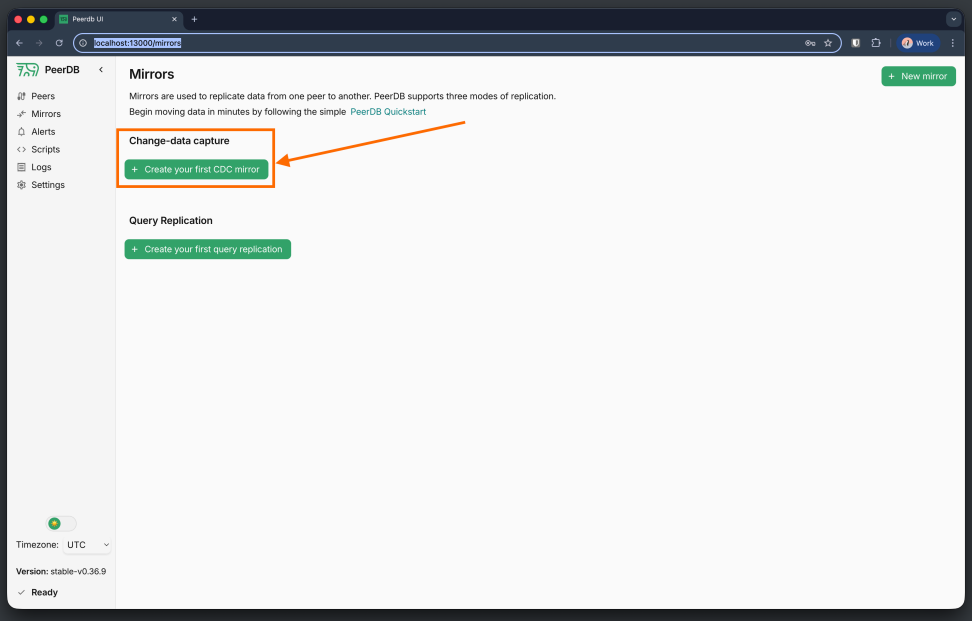
4. Click on **Validate**, it should say the peer is valid.
5. Click on **Create**. You should see the peer created:



Creating a data pipeline between Postgres and ClickHouse

Here, will be creating a **mirror**, which is PeerDB terminology for a data movement pipeline.

1. Head over to http://localhost:13000/mirrors. Click on **+ Create your first CDC mirror**

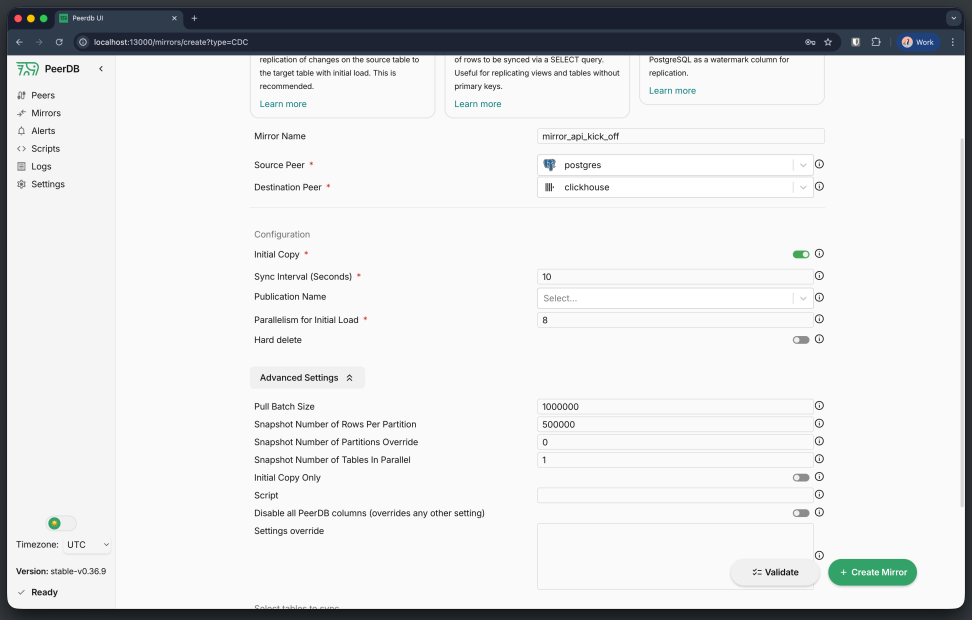


2. You will land on the CDC form by default which is what we want. Enter the following form details:

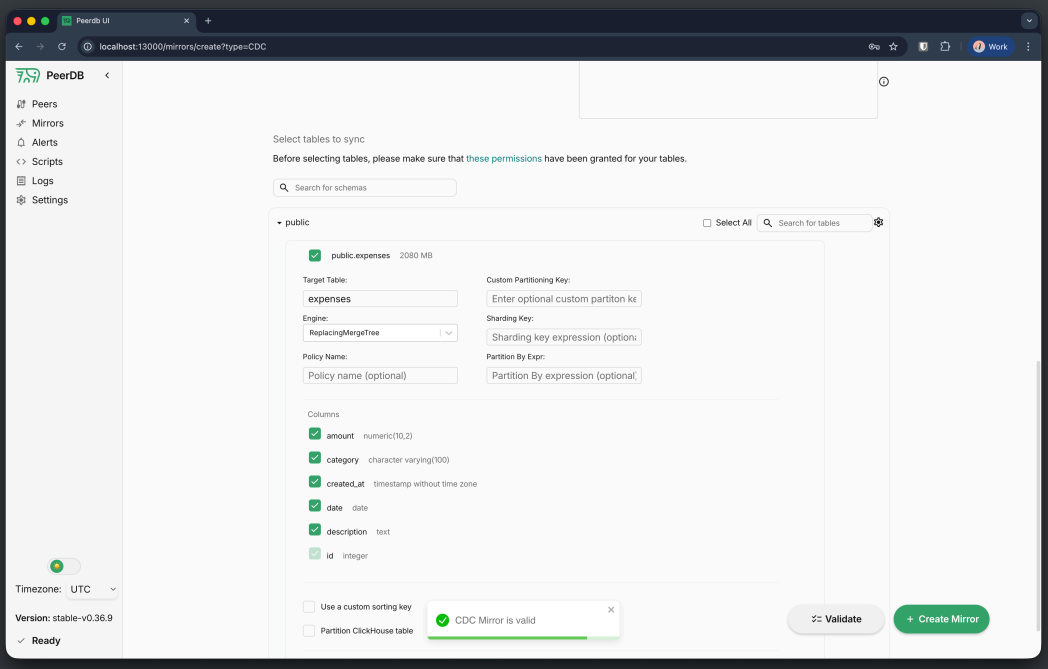
```
Name: mirror_api_kick_off
Source peer: postgres
Destination peer: clickhouse
Sync interval: 10
Parallelism for initial load: 8

Under Advanced settings:
Pull batch size: 1000000
Snapshot Number of Rows Per Partition: 500000
```

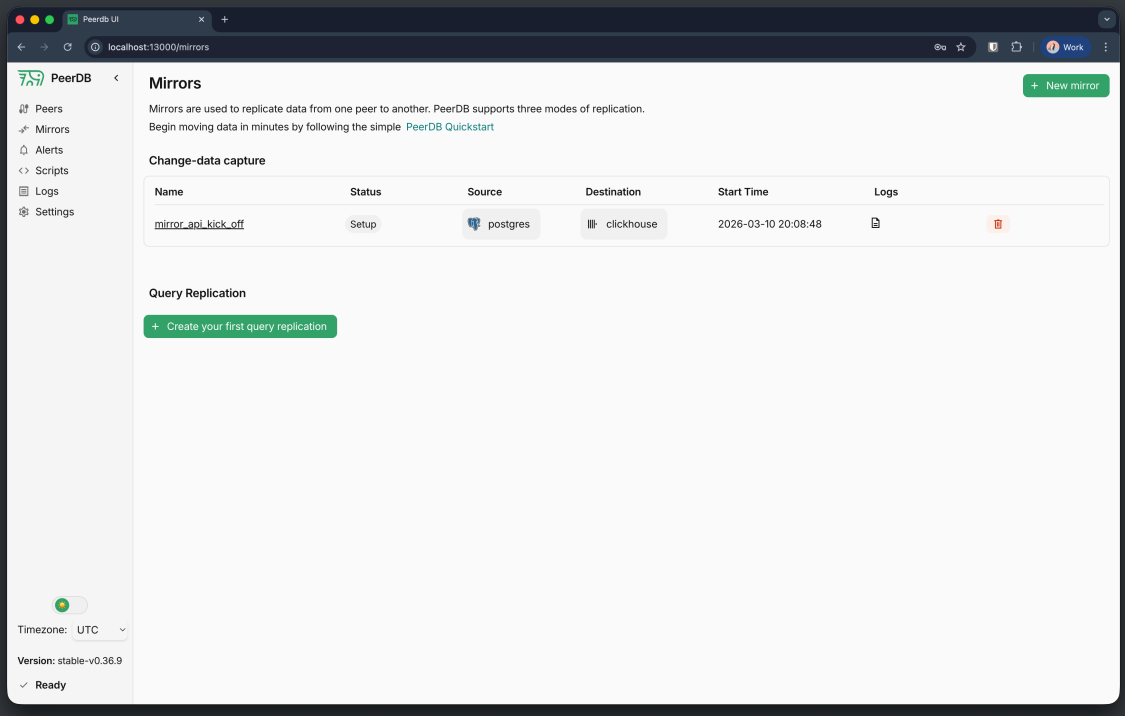
Leave everything else as is. The fields should look like:



3. Now let's select the table we want to sync. Scroll below to the table picker.
4. Expand the `public` schema. You should see the `expenses` table.
5. Select it by clicking on its checkbox.
6. Change the destination table name to **expenses**. (instead of public_expenses)

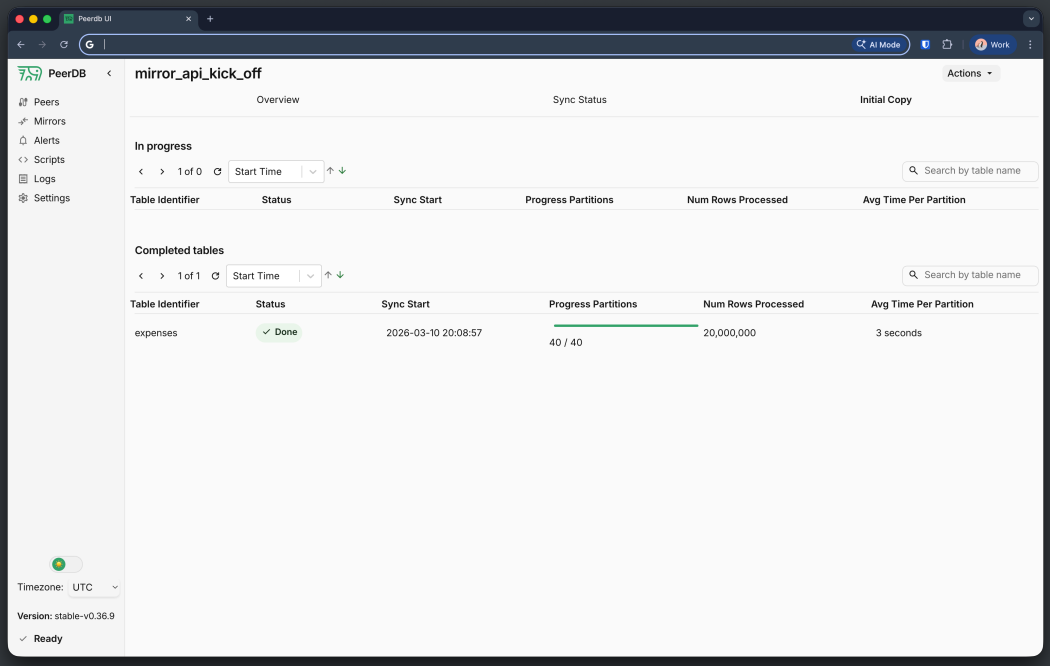


7. Click on **Validate**. It should say the mirror is valid.
8. Click on **Create**. The mirror should get created:



9. Click on the mirror name or go to http://localhost:13000/mirrors/mirror_api_kick_off
10. Click on the **Initial Copy** tab. Refresh here every 10 seconds.

11. Eventually you will see that the initial load is complete:



Examining the replicated data on ClickHouse

1. Shell into ClickHouse server:

```
./run.sh clickhouse
```

2. Connect to the `expense` database:

```
\c expense
```

You should see the following:

```
ch_postgres :) \c expense;

USE expense

Query id: 7e9b97b7-05f2-4ddc-abb3-2924e8ef8702

Connecting to localhost:9000 as user default.
Connected to ClickHouse server version 25.8.18.

Ok.

0 rows in set. Elapsed: 0.119 sec.
```

3. Let's see if the table exists. Run the following:

```
SHOW CREATE TABLE expenses;
```

You should see the following:

```
ch_postgres :) SHOW CREATE TABLE expenses;

SHOW CREATE TABLE expenses

Query id: 824e5021-a232-4496-8b01-12d30cd15d00


┌─statement──────────────────────────────────────────
─┐
1. | CREATE TABLE expense.expenses
  ↴|
   |↳(
  ↴|
   |↳    `id` Int32,
  ↴|
```

```
    |↳    `description` String,
  ↴|
    |↳    `amount` Decimal(10, 2),
    ↴|
    |↳    `category` String,
    ↴|
    |↳    `date` Date32,
    ↴|
    |↳    `created_at` DateTime64(6),
  ↴|
    |↳    `_peerdb_synced_at` DateTime64(9) DEFAULT
now64(),↴|
    |↳    `_peerdb_is_deleted` UInt8,
  ↴|
    |↳    `_peerdb_version` UInt64
    ↴|
    |↳)
  ↴|
    |↳ENGINE = ReplacingMergeTree(_peerdb_version)
    ↴|
    |↳PRIMARY KEY id
    ↴|
    |↳ORDER BY id
  ↴|
    |↳SETTINGS index_granularity = 8192
    |

└──────────────────────────────────────────────────────
┘

1 row in set. Elapsed: 0.006 sec.
```

4. Run a COUNT on the `expenses` table. You should see the 20 million rows replicated:

```
ch_postgres :) SELECT COUNT(*) FROM expenses;
```

```
SELECT COUNT(*)
FROM expenses

Query id: 45743dc4-e8de-499d-bb80-a655f9cc09cc

   ┌──COUNT()─┐
1. │ 20000000 │ -- 20.00 million
   └──────────┘

1 row in set. Elapsed: 0.047 sec.

ch_postgres :)
```

5. We can also take a peek at the data. Run:

```
SELECT * FROM expenses LIMIT 5;
```

You should see something like:

```
ch_postgres :) SELECT * FROM expenses LIMIT 5;

SELECT *
FROM expenses
LIMIT 5

Query id: 8f07c573-9661-43d4-8569-b2490824df54


┌─id─┬─description─────────┬──amount─┬─category─────────
─┬──────date─┬──────────────────created_at─┬─────────────
─_peerdb_synced_at─┬─_peerdb_is_deleted─┬─_peerdb_versio
n─┐
1. │  1 | Travel guide        | 1141.04 | Travel
   | 2025-06-03 | 2026-03-10 19:38:43.980938 |
2026-03-10 20:09:00.422000000 |                  0 |
          0 |
```

```
2. |  2 | Physical therapy    |  496.34 | Healthcare
   | 2025-01-16 | 2026-03-10 19:38:43.980938 |
2026-03-10 20:09:00.422000000 |                  0 |
          0 |
3. |  3 | Coffee              |   59.18 | Food & Dining
    | 2025-12-22 | 2026-03-10 19:38:43.980938 |
2026-03-10 20:09:00.422000000 |                  0 |
          0 |
4. |  4 | Parking fee         |  132.49 | Transportation
   | 2026-02-05 | 2026-03-10 19:38:43.980938 |
2026-03-10 20:09:00.422000000 |                  0 |
          0 |
5. |  5 | Credit card payment |   483.7 | Bills &
Utilities | 2024-10-24 | 2026-03-10 19:38:43.980938 |
2026-03-10 20:09:00.422000000 |                  0 |
          0 |

└────┴──────────────────────┴─────────┴──────────────────
─┴─────────────┴────────────────────────────┴────────────
──────────────────┴────────────────────┴────────────────
──┘

5 rows in set. Elapsed: 0.081 sec.

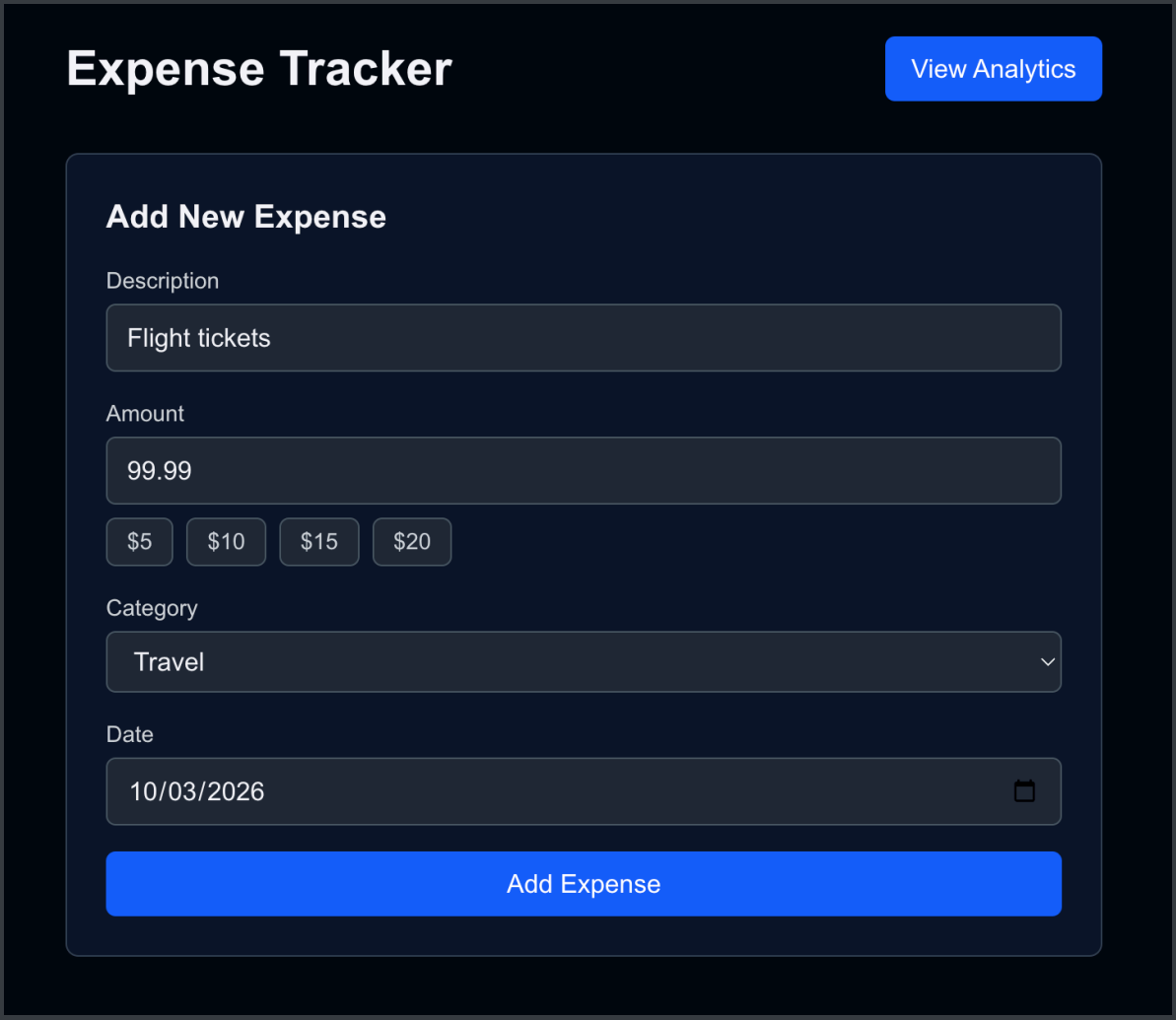
ch_postgres :)
```

Notice how the various String, DateTime data types are consistent with Postgres equivalents.

Create an expense in the app

Now we can finally head over to our NodeJS app which is running on these databases.

1. Head over to http://localhost:18080/
2. Create a new Expense like so:



3. This should have created a new Expense entry in our Postgres table. Let's connect and see:

```
./run.sh postgres
```

```
SELECT * FROM expenses WHERE description = 'Flight tickets';
```

You should see the 2 million and 1th row:

```
postgres=# SELECT * FROM expenses WHERE description =
'Flight tickets';
    id    |  description   | amount | category |    date
   |         created_at
----------+---------------+--------+----------+--------
----+----------------------------
 20000001 | Flight tickets |  99.99 | Travel   |
2026-03-10 | 2026-03-10 20:24:51.004424
(1 row)

postgres=#
```

4. Also, the PeerDB mirror we created earlier should have replicated this row to ClickHouse. Let's connect to ClickHouse:

   ```
   ./run.sh clickhouse
   ```

   Connect to the `expense` database:

   ```
   h_postgres :) \c expense;

   USE expense

   Query id: 7e9b97b7-05f2-4ddc-abb3-2924e8ef8702

   Connecting to localhost:9000 as user default.
   Connected to ClickHouse server version 25.8.18.

   Ok.
   ```

   and query for the same row:

   ```
   SELECT * FROM expenses WHERE description = 'Flight
   tickets';
   ```

   We should see the row here as well thanks to replication:

```
ch_postgres :) SELECT * FROM expenses WHERE description
= 'Flight tickets';

SELECT *
FROM expenses
WHERE description = 'Flight tickets'

Query id: a95a7847-0616-405c-907c-b3b46af38d72


┌───────id─┬─description───┬─amount─┬─category─┬────────
date─┬─────────────────created_at─┬─────────────_peerdb_
synced_at─┬─_peerdb_is_deleted─┬─────_peerdb_version─┐
1. | 20000001 | Flight tickets |  99.99 | Travel   |
2026-03-10 | 2026-03-10 20:24:51.004424 | 2026-03-10
20:25:01.270000000 |                    0 |
1773174291096178743 |

└──────────┴─────────────────┴────────┴──────────┴───────
─────┴────────────────────────────┴────────────────────
──────────┴──────────────────────┴─────────────────────┘
```

That's the magic of PeerDB!

Using the pg_clickhouse extension

Here, we are going to see how we can use `pg_clickhouse` to query ClickHouse directly from a PSQL terminal aka Postgres.

1. Enable the extension:

```
CREATE EXTENSION IF NOT EXISTS pg_clickhouse;
```

2. Create the foreign data wrapper:

```
CREATE SERVER clickhouse_svr FOREIGN DATA WRAPPER
clickhouse_fdw OPTIONS(dbname 'expense', host
'clickhouse');
```

3. Create a user mapping:

```
CREATE USER MAPPING FOR CURRENT_USER SERVER
clickhouse_svr OPTIONS (user 'default', password
'clickhouse');
```

4. Create a schema to store the ClickHouse foreign tables:

```
CREATE SCHEMA IF NOT EXISTS expense_ch;
```

5. Now, let's import the ClickHouse data:

```
IMPORT FOREIGN SCHEMA expense FROM SERVER clickhouse_svr
INTO expense_ch;
```

6. And now let's query the ClickHouse table from Postgres!

```
postgres=# SELECT COUNT(*) FROM expense_ch.expenses;
  count
----------
 20000001
(1 row)
```

7. To see the FDW in action, we can run `EXPLAIN` commands on the above COUNT query:

```
EXPLAIN SELECT COUNT(*) FROM expense_ch.expenses;
EXPLAIN SELECT COUNT(*) FROM public.expenses;
```

   You'll see the first one says foreign scan:

```
```

```
postgres=# EXPLAIN SELECT COUNT(*) FROM
expense_ch.expenses;
                        QUERY PLAN
-----------------------------------------------------
 Foreign Scan  (cost=1.00..0.20 rows=1000 width=8)
   Relations: Aggregate on (expenses)
(2 rows)

postgres=# EXPLAIN SELECT COUNT(*) FROM public.expenses;
                                              QUERY PLAN
-------------------------------------------------------------------
-------------------------------------
 Finalize Aggregate  (cost=296426.28..296426.29 rows=1
width=8)
   ->  Gather  (cost=296426.06..296426.27 rows=2
width=8)
         Workers Planned: 2
         ->  Partial Aggregate
 (cost=295426.06..295426.07 rows=1 width=8)
               ->  Parallel Seq Scan on expenses
 (cost=0.00..274590.85 rows=8334085 width=0)
 JIT:
   Functions: 4
   Options: Inlining false, Optimization false,
Expressions true, Deforming true
(8 rows)
```

Conclusion

The walkthrough is complete!

We saw how PeerDB connected Postgres and ClickHouse -- two very different databases. We then saw how pg_clickhouse then enables a one-stop shop query layer, unifying this data stack.